Week 6: Mixture modelling

Emma James

1 Introduction

> **Learning objectives**
>
> As a reminder, the learning objectives for this week are:
>
> 1. Describe the difference between variable-centred and person-centred approaches
> 2. Explain how categorical latent variables can be used to identify unobserved subpopulations
> 3. Fit and compare latent profile models that vary in their constraints and k-classes
> 4. Interpret model output to identify subpopulations
> 5. Discuss factors that affect the implementation and generalisability of mixture models

This week we've taken a *person-centred* approach to analysing data: what can we learn about heterogeneity within the population? In today's practical, we will continue to use latent profile analysis as our example of mixture modelling—i.e., we will use continuous variables as indicators of different subpopulations. We will start with a built-in dataset of student survey responses, before analysing a dataset of early mathematical skills from a recently published paper.

We will use the *tidyLPA* package to fit these models. A nice advantage of this package is that it is designed to align with the *tidyverse* framework, and so we can use pipes to progress between each modelling step.

---

2 Setting up

2.1 Packages

Today's practical will use the following packages. You will need to install each one, before loading them at the top of your script.

```
library(tidyverse)  # for data wrangling
library(readxl)     # for importing data in Excel format
library(tidyLPA)    # for latent profile analysis

options(scipen = 999)
```

You might also need to manually install the `mix` package so that it is available for some of the *tidyLPA* functions to call upon.

---

3 Part A: Core example

3.1 Introduction to the dataset

We will start by using the built-in dataset from the tidyLPA package, `pisaUSA15`. PISA stands for the Programme for International Student Assessment, and represents an international survey of 15-year-old school pupils. This subset of data is from the 2015 survey and includes questions about US students' attitudes towards school.

This is a real dataset with some missingness that would need to be considered in a full analysis. We'll come back to these issues later in the module, but for the purpose of today's practical we will exclude cases with missing data. Let's also cut back to a slightly smaller sample to speed up model fitting. We can load the dataset and complete these preprocessing steps as follows:

```r
# Load and preprocess data
pisa_dat <- pisaUSA15 %>%
  drop_na() %>%
  slice_head(n = 2000)
```

Access the help files for the original dataset and inspect the variable descriptions (e.g., using `help()` or `?`). Use R code to answer the following questions about the dataset:

1. How many variables are there?
2. What is the mean and range for each variable?
3. How do the variables correlate with each other?

```r
# Number of pupils (rows in dataset)
nrow(pisa_dat)

# Number of variables (columns in dataset)
ncol(pisa_dat)

# Variable descriptions
summary(pisa_dat)

# Correlations
cor(pisa_dat, use = "pairwise.complete.obs") %>%
  round(2)
```

3.2 Class enumeration procedure

In latent profile analysis, we are asking how many (k) classes are optimal to account for patterns of performance in the data. We are typically conducting this type of analysis under the assumption that there is heterogeneity in the population (i.e., more than one group), but we rarely have strong hypotheses about the precise number and nature of the classes. A typical approach is therefore to fit several models of different k-classes and compare model fit.

3.2.1 Fit models

Let's start here with the simplest possible model. This *class-invariant* model assumes that the only thing that differs between classes is their mean score on the items. We assume that the variance of these scores is equal across classes, and don't estimate any relationships (covariances) between the items.

Use the `estimate_profiles()` function as follows to fit 5 different models, increasing from 1 class to 5 classes. We specify that the variances should be equal across classes, and that covariances should be constrained to zero. This might take a minute or two to run—be patient!

```
# Fit models
pisa_m1 <- pisa_dat %>%
  estimate_profiles(n_profiles = c(1:5),  # fit models with k = 1-5
                    variances = "equal",
                    covariances = "zero")
```

3.2.2 Identify model of best fit

Now that we have fitted the models, we can use the fit indices to help us to determine the optimal model to account for the data. Extract your fit indices as follows, then print them in the console or open them in the viewer.

```
# Extract fit indices
pisa_m1_fit <- get_fit(pisa_m1)
```

Each row provides fit indices for one of the models you fitted (so five in total, because we increased the number of k classes from 1 to 5). *TidyLPA* provides us with lots of information by default, so let's focus on a few key elements to start with:

- **Model:** This number reflects the model structure (i.e., whether variances and covariances are allowed to vary). The labels are found on the vignette, but they are all the same here as we only fitted one model type.
- **Classes:** This is the number (k) of classes fitted in each model. This tells us which of our different k-class models each row corresponds too.
- **AIC - SABIC:** These are all different types of fit indices, which differ in how they penalise model complexity and how they perform with datasets of different sizes. More on these below!
- **BLRT_p:** This tells us whether each model is significantly better than the k-1 model. In this set of models, this doesn't look to be particularly informative (i.e., each additional class always leads to a significant increase in model fit, $p < .05$).

Let's look at the three most commonly used fit indices suggested by Masyn (2013): the Bayesian Information Criterion (**BIC**), the Consistent Akaike's Information Criterion (**CAIC**), and Approximate Weight of Evidence Criterion (**AWE**). In each of these, the best model is the one with the smallest value. Which model do they each identify as the best one?

> **Top tip! Plotting fit indices**
>
> It can be helpful to plot these to inspect them. Unfortunately *tidyLPA* doesn't include any built-in functions for this, but I've drafted some *ggplot* code that you're welcome to play around with here. In each of these, we can see an "elbow" at the 4-class model—the improvement in fit beyond this point is much smaller.

```
# Take output of get_fit()
pisa_m1_fit %>%

  # Select k-class variable indices of interest
  select(c(Model, Classes, BIC, CAIC, AWE)) %>%

  # Turn this into a long dataset to implement panel plotting
  pivot_longer(c(BIC:AWE), names_to = "fit_index", values_to = "value") %>%

  # Create ggplot object
  ggplot(aes(x = Classes, y = value, colour = as.factor(Model))) +

  # Tidy plotting theme
  theme_classic() +

  # Plot data points and path
  geom_point(shape = 4) +
  geom_path() +

  # Create separate panels/plots for each fit index
  facet_wrap(vars(fit_index), scales = "free")
```

In this example, it's clear that the 4-class model is the best option by most metrics. We can also see that this model has a good level of entropy (0.94), indicating good separation between the classes.

3.3 Widening the search space

Thus far, our models have not accounted for covariances between the items (i.e., we fixed the covariances to 0). Estimating these increases the complexity of the model (you will see that there are more parameters in the output), but gives the model more information for working out the classes. Let's add in this estimation, but still keep them equal across classes[1]. Extract the fit indices as before.

[1] This represents Model 3 in the *tidyLPA* documentation.

```
# Fit model
pisa_m3 <- pisa_dat %>%
  estimate_profiles(c(1:5),  # fit models with k = 1-5
                    variances = c("equal"),
                    covariances = c("equal"))

# Extract fit indices
pisa_m3_fit <- get_fit(pisa_m3)
```

Inspect your new model fit indices. Does the BLRT test tell us anything this time around about improvements in model fit? Which *k*-class model here is the best one by BIC, CAIC, and AWE?

3.4 Choosing the final model

In Steps 3.2 and 3.3, we identified two candidate models to choose between (one from each specification). The final model to proceed with can be decided based on several factors, including:

- **Fit indices:** Using the same indices we paid attention to above, is one a much better fit than the other? Plotting can be helpful again here for comparison (code below if you would like to do this).
- **Entropy:** Does one model provide more distinct classes than the other?
- **Interpretability/theoretical judgement:** We are not experts in this dataset so won't try this one here, but we might look at the final classes estimated by each model and consider whether they are meaningful (as in Step 3.5). We tend to avoid very small classes unless they are clearly distinct and align with theoretical expectations.
- **Simplicity!** If the models look comparably good on all other metrics, we tend to prefer a simpler model (i.e., one with fewer parameters) that is more likely to generalise.

```
# Join fit indices in one dataframe
pisa_fit_full <- pisa_m1_fit %>%
  bind_rows(pisa_m3_fit)

# Plot
pisa_fit_full %>%
  select(c(Model, Classes, BIC, CAIC, AWE)) %>%
  pivot_longer(c(BIC:AWE), names_to = "fit_index", values_to = "value") %>%
  ggplot(aes(x = Classes, y = value, group = Model, colour = as.factor(Model))) +
  theme_classic() +
  geom_path() +
  facet_wrap(vars(fit_index), scales = "free")
```

Choose your preferred model to take forward to the final step. Note that the two model specifications were more than enough to consider for the practical today. However, in reality, we might continue to fit more models with increasingly complex variance/covariance structures and compare the best candidate models from each.

3.5 Inspecting the final model

Hurrah, we've arrived at our final model! I've chosen Model 1 here with 4 classes, though the 4-class model with covariances estimated could also have been a good choice. Now for the big reveal: what does it show us about subpopulations in the dataset?

The easiest way to interpret this is to plot how each class performs on the different measures. We can do this using a built-in function:

```
# Plot profiles
plot_profiles(pisa_m1$model_1_class_4,
              rawdata = FALSE,
              sd = FALSE,
              ci = 0.95)  # default but can also switch to false
```

I've switched off the plotting for raw data (messy with large datasets, slow to plot) and standard deviations (hard to read alongside the confidence intervals), but feel free to play around with these options.

Have a look at the different classes from the plot. It seems like Class 4 for example show the highest school interest and enjoyment, but are the lowest on motivation and self-efficacy. By comparison, Class 2 shows fairly middle-range responses across the different measures. It's standard practice to come up with descriptive labels for these classes when communicating the findings.

If we wanted to extract these specific model estimates for reporting, we could use the `get_estimates()` function. We can also extract the class assignment for each individual participant as follows:

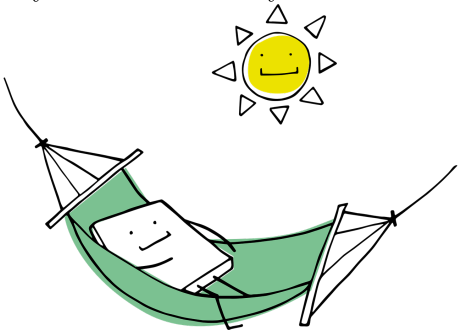
```
# Extract data and round probabilities to 2 d.p.
pisa_class_probs <- get_data(pisa_m1$model_1_class_4) %>%
  mutate(across(CPROB1:CPROB4, round, 2))

# Preview dataframe
head(pisa_class_probs)
```

You should see that each participant has a probability of being assigned to each of the four classes (`CPROB1-4`), with their highest probability class assignment presented in the final `Class` column. For most participants this assignment is pretty clear cut, which is an indication of clear class separation in the model. However, we see for the sixth participant that there is fairly equal probability that they could be assigned to Class 2 or 3. We can see from the profile plots above that these two classes are fairly similar on 2-3 of the 4 measures, so this uncertainty is perhaps to be expected.

Finally, you can also use the extracted data to work out the proportion of individuals assigned to each class:

```
# Calculate proportions
pisa_class_probs %>%
  group_by(Class) %>%
  summarise(n = n(),
            proportion = n/nrow(pisa_class_probs))
```

> Take a break!
>
> If you haven't already taken a break today, we suggest you do so before you start the next section.
>
> 

4 Part B: Apply your knowledge

4.1 Introduction to the dataset

Now let's take a look at a dataset from a recently published study. In this paper by Silla et al. (2026), they examined different aspects of children's fraction knowledge. They wanted to know whether there are distinct patterns of early fraction knowledge that characterise different children, and how this predicted other cognitive skills and maths achievement. Their data and an open-access preprint of their manuscript is available on the OSF. *[Note I had some problems importing this into R directly, so I have shared the data on the VLE].*

Read in the data and select the relevant variables for analysis

```
# Load data
fraction_dat <- read_xlsx("./data/FractionItemLevel_FallSpring_OSF_20240907.xlsx") %>%
  select(NONEQSFP:SYMWRFP)
```

The dataset contains the following variables:

- **NONEQSFP:** Non-symbolic - Equal sharing
- **NONEQFP:** Non-symbolic - Equivalence
- **NONITFP:** Non-symbolic - Iterating (Adding) equal parts
- **SYMWORFP:** Symbolic - Mapping fraction words to symbols
- **SYMWRFP:** Symbolic - Fraction notation

Use R code to answer the following questions about the dataset:

- How many children completed the tasks?
- What is the mean and range of each task?

- Is there any missing data to address?
- How do the tasks correlate with each other?

```
# Number of children
nrow(fraction_dat)

# Mean and range - this would also flag any NAs
summary(fraction_dat)

#... or if you want a different method - tells us whether the number of complete cases is the sam
fraction_dat %>%
  filter(complete.cases(.)) %>%
  nrow()

# Correlations
cor(fraction_dat) %>%
  round(digits = 2)
```

4.2 Conduct latent profile analysis

Use your new skills to establish how many different profiles of fraction knowledge are observable in the dataset and what those profiles look like. You can do this via the following steps:

1. Fit models with k = 1-5, with equal variances and no covariances (Model 1)[2].
2. Select the best candidate k-class model(s) for this model specification, based on the BIC, CAIC, and AWE fit indices and the bLRT test.
3. Fit models with k = 1-5, with equal variances and equal covariances (Model 3).
4. Select the best candidate k-class model(s) for this model specification, based on the BIC, CAIC, and AWE fit indices and the bLRT test.
5. Compare the best models against each other, and weigh up the evidence in favour of each. Which model do you choose?
6. Plot the profiles from your final model, and describe each one verbally.

```
# Fit model 1 (most constrained)
fraction_m1 <- fraction_dat %>%
  estimate_profiles(c(1:5),
                    variances = "equal",
                    covariances = "zero")

# Inspect fit
get_fit(fraction_m1)
## bLRT favours 4-class
```

[2]You will likely get a warning that some of the variables may not be strictly continuous. We will ignore this for now given that *tidyLPA* does not offer an alternative estimation algorithms.

```
## AWE favours 2-class
## BIC favours 3-class
## CAIC favours 3-class

# Fit model 3 (including covariances)
fraction_m3 <- fraction_dat %>%
  estimate_profiles(c(1:5),
                    variances = "equal",
                    covariances = "equal")

# Inspect fit
get_fit(fraction_m3)

## bLRT favours 3-class
## AWE favours 2-class
## BIC favours 3-class
## CAIC favours 3-class

# Choose best options
## 3-class looks promising across both specifications
## Plot for easy inspection

fraction_fit_full <- get_fit(fraction_m1) %>%
  bind_rows(get_fit(fraction_m3))

fraction_fit_full %>%
  select(c(Model, Classes, BIC, CAIC, AWE)) %>%
  pivot_longer(c(BIC:AWE), names_to = "fit_index", values_to = "value") %>%
  ggplot(aes(x = Classes, y = value, group = Model, colour = as.factor(Model))) +
  theme_classic() +
  geom_path() +
  facet_wrap(vars(fit_index), scales = "free")

## simpler model preferred
```

---

5 Part C: Challenge yourself!

Today's practical has already been pretty challenging! If you want to push yourself further, take a look at the *tidyLPA* vignette and see which other model specifications you can fit with the *tidyLPA* package alone (i.e., using *mclust* rather than *MPlus*). What happens when you try to run them on the datasets that we used today?

---

6 Summary

6.1 Recap

Today's practical showed us an example of a mixture modelling approach in action. We saw how a latent variable with different numbers *(k)* of categories can be fitted and compared to identify unobserved subpopulations in the data. We also varied the constraints of the model, and saw how we can use fit statistics to identify the best model that balances precision and parsimony. Finally, we inspected the characteristics of each profile in the final model, and extracted the probabilistic class membership for individuals in the dataset.

6.2 Further learning

This week's core reading can be found on the reading list as usual. This is one of—indeed maybe even *the*—most complex analyses we will cover in this module, and there's a great Quantitude podcast episode if you're struggling to get your head around the main ideas.

There is also a DataCamp course on Mixture Models in R. This course uses the *flexmix* package rather than *tidyLPA*, which has a slightly steeper learning curve but is more flexible to different types of mixture models. You might want to consider this to build your experience or if you are likely to run this type of analysis in future.

! Feedback!

These practical sessions are new this year. Please take a moment to rate the difficulty and length of these practical activities via this survey, and let us know of any issues you encountered or aspects you didn't understand (positive feedback is also welcome!). You will need to be logged in with your university account to respond, but your submission will be anonymous.
For more general questions about the practical, please post them on the VLE Discussion Board.